

Montando um testbed 5G SA

Este tutorial apresenta o passo a passo para implantação de uma rede de comunicação móvel 5G utilizando USRP B200, srsRAN e Open5GS. Ao decorrer das próximas páginas você aprenderá a instalar os softwares necessários, dependências e modificar arquivos de configuração com os principais parâmetros da rede 5G.

Esse tutorial foi elaborado e validado no Laboratório InComm da EMC-UFG, sendo orientado pelo Prof. Dr. Flávio Geraldo Coelho Rocha, contando com a colaboração do Doutorando Maykon Renan Pereira da Silva e efetivada pelo discente Ezequiel Reis Tavares. Agradecemos ao Prof. Dr. Kleber Vieira Cardoso por conceder o SDR e o SIM card que foram necessários para implementação deste testbed.

O documento está organizado em seções, conforme o sumário apresentado a seguir.

- 1. Instalando o driver da USRP**
- 2. Instalando srsRAN_Project**
- 3. Instalando Open5GS a partir do código fonte**
- 4. Instalando o gravador de simcard**
- 5. Configurando e executando**
- 6. Executando os binários e usando a rede configurada**

Hardware utilizado:

1. Servidor equipado com processador Xeon E5-1630, 16 GB de memória RAM, 1 TB de armazenamento, com sistema operacional Ubuntu 22.04.03 LTS;
2. SDR (USRP-B200);
3. SIM card (Open-cells e Sysmocom);
4. Smartphone com acesso ao 5G (Android).

Obs.: o ideal é que o item 1 seja um hardware similar ou superior e contenha uma entrada USB 3.0 ou superior.

Recomendo que ao início de cada uma das instalações utilize:

```
sudo apt-get update  
cd ~
```

Informativo: o primeiro comando remove versões antigas de pacotes instalados ou atualizações no sistema que não são mais necessárias e o segundo serve para acessar o seu diretório “home” do Linux..

1. Instalando o driver da USRP

Instale as dependências necessárias:

```
sudo apt-get install autoconf automake build-essential ccache cmake cpufrequtils doxygen  
ethtool g++ git inetutils-tools libboost-all-dev libncurses5 libncurses5-dev libusb-1.0-0  
libusb-1.0-0-dev libusb-dev python3-dev python3-mako python3-numpy python3-requests  
python3-scipy python3-setuptools python3-ruamel.yaml
```

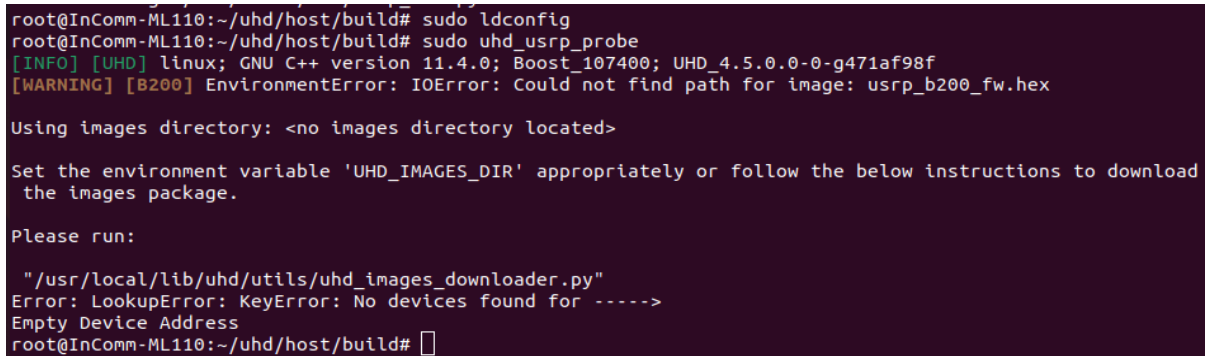
A fonte UHD é armazenada em um repositório git. Logo, execute o seguinte comando no terminal:

```
git clone https://github.com/EttusResearch/uhd.git
```

Construir e instalar:

```
cd uhd/host  
mkdir build  
cd build  
cmake ../  
make  
sudo make install  
sudo ldconfig  
sudo uhd_usrp_probe
```

Nota 01: é normal que o terminal apresente isto mesmo com o dispositivo conectado, pois faltam as imagens.



```
root@InComm-ML110:~/uhd/host/build# sudo ldconfig  
root@InComm-ML110:~/uhd/host/build# sudo uhd_usrp_probe  
[INFO] [UHD] linux; GNU C++ version 11.4.0; Boost_107400; UHD_4.5.0.0-g471af98f  
[WARNING] [B200] EnvironmentError: IOError: Could not find path for image: usrp_b200_fw.hex  
  
Using images directory: <no images directory located>  
  
Set the environment variable 'UHD_IMAGES_DIR' appropriately or follow the below instructions to download  
the images package.  
  
Please run:  
  
"/usr/local/lib/uhd/uhd_images_downloader.py"  
Error: LookupError: KeyError: No devices found for ----->  
Empty Device Address  
root@InComm-ML110:~/uhd/host/build#
```

Para contornar tal problema execute o comando:

```
sudo /usr/local/lib/uhd/uhd_images_downloader.py  
sudo ldconfig  
cd
```

Informativo: o primeiro comando faz a transferência das respectivas imagens no diretório adequado, o segundo cria links e refaz o índice das bibliotecas dinâmicas do arquivo, já o terceiro serve para inspecionar o equipamento de rádio conectado e retornar suas informações técnicas e configurações.

Tendo êxito o terminal ficará assim:

```
[INFO] [UHD] linux; GNU C++ version 11.4.0; Boost_107400; UHD_4.5.0.0-g471af98f
[INFO] [B200] Loading firmware image: /usr/local/share/uhd/images/usrp_b200_fw.hex...
[INFO] [B200] Detected Device: B200
[INFO] [B200] Loading FPGA image: /usr/local/share/uhd/images/usrp_b200_fpga.bin...
[INFO] [B200] Operating over USB 3.
[INFO] [B200] Detecting internal GPSDO....
[INFO] [GPS] No GPSDO found
[INFO] [B200] Initialize CODEC control...
[INFO] [B200] Initialize Radio control...
[INFO] [B200] Performing register loopback test...
[INFO] [B200] Register loopback test passed
[INFO] [B200] Setting master clock rate selection to 'automatic'.
[INFO] [B200] Asking for clock rate 16.000000 MHz...
[INFO] [B200] Actually got clock rate 16.000000 MHz.
```

```
uhd_config_info --version
```

2. Instalando srsRAN_Project

Instale as dependências necessárias:

```
sudo apt-get install cmake make gcc g++ pkg-config libfftw3-dev libmbedtls-dev libsctp-dev
libyaml-cpp-dev libgtest-dev
```

A fonte srsRAN também é armazenada em um repositório git. Logo, basta fazer o mesmo que fizeste anteriormente:

```
git clone https://github.com/srsRAN/srsRAN_Project.git
```

Construir e instalar:

```
cd srsRAN_Project
mkdir build
cd build
cmake ../
make -j $(nproc)
make test -j $(nproc)
```

```
100% tests passed, 0 tests failed out of 4643
```

```
Label Time Summary:
```

```
NO_MEMCHECK      = 1.28 sec*proc (1 test)
adt               = 8.40 sec*proc (358 tests)
asn1              = 1.39 sec*proc (30 tests)
cell_meas_manager = 0.12 sec*proc (11 tests)
cu               = 0.08 sec*proc (1 test)
cu_cp            = 4.71 sec*proc (113 tests)
cu_cp_ue_manager = 0.58 sec*proc (17 tests)
cu_up           = 0.34 sec*proc (15 tests)
cu_up_processor  = 0.08 sec*proc (1 test)
du              = 0.08 sec*proc (1 test)
du_high         = 6.60 sec*proc (3 tests)
du_high|tsan    = 0.60 sec*proc (1 test)
du_manager      = 2.06 sec*proc (117 tests)
du_processor    = 1.33 sec*proc (38 tests)
e1ap            = 0.80 sec*proc (33 tests)
e1ap_cu_cp      = 0.43 sec*proc (21 tests)
e1ap_cu_up      = 0.28 sec*proc (10 tests)
e2ap            = 2.88 sec*proc (23 tests)
ecpri           = 0.07 sec*proc (8 tests)
ethernet        = 7.63 sec*proc (171 tests)
f1ap            = 1.79 sec*proc (61 tests)
```

Para terminar a instalação:

```
sudo make install
```

```
sudo ldconfig
```

3. Instalando Open5GS a partir do código fonte

➤ Obtendo MongoDB

Informativo: o MongoDB funciona de maneira diferente dos bancos de dados relacionais tradicionais, onde os dados são armazenados em tabelas com relações predefinidas. Os dados são armazenados em documentos, que são estruturados em formato BSON e organizados em coleções. Cada documento contém todas as informações relevantes em um único lugar, o que permite uma rápida recuperação de dados e uma melhor performance em ambientes distribuídos. Ademais, ele é um requisito obrigatório para a utilização do Open5GS.

Importe a chave pública usada pelo sistema de gerenciamento de pacotes utilizando os seguintes comandos:

```
sudo apt install gnupg
```

```
curl -fsSL https://pgp.mongodb.com/server-6.0.asc | sudo gpg -o  
/usr/share/keyrings/mongodb-server-6.0.gpg --dearmor
```

Crie o arquivo de lista /etc/apt/sources.list.d/mongodb-org-6.0.list para a versão Ubuntu 22.04:

```
echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-6.0.gpg]
https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/6.0 multiverse" | sudo tee
/etc/apt/sources.list.d/mongodb-org-6.0.list
```

Por fim, instale os pacotes:

```
sudo apt update
sudo apt install -y mongodb-org
```

Verifique se o software está em execução:

```
service mongod status
```

Se `/usr/bin/mongod` não estiver em execução:

```
sudo systemctl start mongod
```

Para iniciá-lo automaticamente na inicialização do sistema:

```
sudo systemctl enable mongod
```

➤ **Obtendo OPen5GS**

Instale as dependências para construir o código-fonte:

```
sudo apt install python3-pip python3-setuptools python3-wheel ninja-build build-essential
flex bison libsctp-dev libgnutls28-dev libgcrypt-dev libssl-dev libidn11-dev libmongoc-dev
libbson-dev libyaml-dev libnghttp2-dev libmicrohttpd-dev libcurl4-gnutls-dev libnghttp2-dev
libtins-dev libtalloc-dev meson
```

A fonte OPen5GS também é armazenada em um repositório git. Logo, basta fazer o mesmo que fizeste anteriormente:

```
git clone -b v2.6.6 https://github.com/open5gs/open5gs.git
```

Construir e instalar:

```
cd open5gs
meson build --prefix=`pwd`/install
```

Informativo: o meson é um sistema de compilação, de código aberto, que aumenta a eficiência do processo que está sendo utilizado.

```
Configuration file for the 'meson' build system
Message:
  prefix: /root/open5gs/install
  libdir: /root/open5gs/install/lib/x86_64-linux-gnu
  bindir: /root/open5gs/install/bin
  sysconfdir: /root/open5gs/install/etc
  localstatedir: /root/open5gs/install/var
  build location: /root/open5gs/build
  source location: /root/open5gs
  compiler: gcc
  debugging support: debug

Build targets in project: 95

open5gs 2.6.6

Subprojects
  freeDiameter : YES 1 warnings
  prometheus-client-c: YES

User defined options
  prefix : /root/open5gs/install

Found ninja-1.10.1 at /usr/bin/ninja
```

`ninja -C build`

Verifique se a compilação está correta:

`./build/tests/attach/attach`

```
root@InComm-ML110:~/open5gs# ./build/tests/attach/attach
s1setup-test : SUCCESS
simple-test : SUCCESS
guti-test : SUCCESS
auth-test : SUCCESS
idle-test : SUCCESS
emm-status-test : SUCCESS
ue-context-test : SUCCESS
reset-test : SUCCESS
issues-test : SUCCESS
crash-test : SUCCESS
All tests passed.
```

`cd build`

`meson test -v`

```
Ok: 13
Expected Fail: 0
Fail: 0
Unexpected Pass: 0
Skipped: 0
Timeout: 0

Full log written to /root/open5gs/build/meson-logs/testlog.txt
```

Obs.: se o seu terminal apresentou algum erro, refaça os passos antes de prosseguir.

```
cd build
ninja install
ldconfig
cd ../
```

➤ Instalando a UI da Web do Open5GS

Baixe e importe a chave Nodestorage GPG usando:

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://deb.nodesource.com/gpgkey/nodesource-repo.gpg.key | sudo gpg --dearmor
-o /etc/apt/keyrings/nodesource.gpg
```

Crie o repositório deb:

```
NODE_MAJOR=20
echo "deb [signed-by=/etc/apt/keyrings/nodesource.gpg]
https://deb.nodesource.com/node_${NODE_MAJOR}.x nodistro main" | sudo tee
/etc/apt/sources.list.d/nodesource.list
```

Execute a atualização e instale:

```
sudo apt update
sudo apt install nodejs -y
```

Por fim, instale a WebUI do Open5GS:

```
curl -fsSL https://open5gs.org/open5gs/assets/webui/install | sudo -E bash -
```

```
[Install]
WantedBy=multi-user.target
EOF
+ systemctl daemon-reload
+ systemctl enable open5gs-webui
Created symlink /etc/systemd/system/multi-user.target.wants/open5gs-webui.service → /lib/systemd/system/open5gs-webui.service.
+ deb-systemd-invoke start open5gs-webui

## Default Administrator Account [Username:admin, Password:1423]...

+ mongosh open5gs ./open5gs-2.6.6/docs/assets/webui/mongo-init.js
Current Mongosh Log ID: 652ec8f47ad3ab815b35c174
Connecting to:      mongodb://127.0.0.1:27017/open5gs?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.0.1
Using MongoDB:      6.0.10
Using Mongosh:       2.0.1

For mongosh info see: https://docs.mongodb.com/mongodb-shell/

-----
The server generated these startup warnings when booting
2023-10-17T12:30:47.710-03:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongo
db.org/core/prodnotes-filesystem
2023-10-17T12:30:49.253-03:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestrict
ed
2023-10-17T12:30:49.254-03:00: vm.max_map_count is too low
-----
Loading file: ./open5gs-2.6.6/docs/assets/webui/mongo-init.js
+ rm -rf ./open5gs-2.6.6
```

4. Instalando o gravador de simcard

Antes de conectar o dispositivo na porta usb:

`lsusb`

```
root@incomm-ProLiant-ML110-Gen9:/home/ezequiel# lsusb
Bus 002 Device 002: ID 8087:8002 Intel Corp. 8 channel internal hub
Bus 002 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 003 Device 001: ID 1d6b:0001 Linux Foundation 1.1 root hub
Bus 001 Device 002: ID 8087:800a Intel Corp. Hub
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 005 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 004 Device 008: ID 04ca:00bd Lite-On Technology Corp. Dell Wireless Device
Bus 004 Device 002: ID 0424:2660 Microchip Technology, Inc. (formerly SMSC) Hub
Bus 004 Device 006: ID 04b3:3025 IBM Corp. NetVista Full Width Keyboard
Bus 004 Device 005: ID 046d:c05a Logitech, Inc. M90/M100 Optical Mouse
Bus 004 Device 004: ID 062a:5918 MosArt Semiconductor Corp. 2.4G Keyboard Mouse
Bus 004 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

Após conectar o dispositivo:

`lsusb`

```
root@incomm-ProLiant-ML110-Gen9:/home/ezequiel# lsusb
Bus 002 Device 002: ID 8087:8002 Intel Corp. 8 channel internal hub
Bus 002 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 003 Device 001: ID 1d6b:0001 Linux Foundation 1.1 root hub
Bus 001 Device 002: ID 8087:800a Intel Corp. Hub
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 005 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 004 Device 009: ID 067b:2303 Prolific Technology, Inc. PL2303 Serial Port / Mobile Action MA-8910P
Bus 004 Device 008: ID 04ca:00bd Lite-On Technology Corp. Dell Wireless Device
Bus 004 Device 002: ID 0424:2660 Microchip Technology, Inc. (formerly SMSC) Hub
Bus 004 Device 006: ID 04b3:3025 IBM Corp. NetVista Full Width Keyboard
Bus 004 Device 005: ID 046d:c05a Logitech, Inc. M90/M100 Optical Mouse
Bus 004 Device 004: ID 062a:5918 MosArt Semiconductor Corp. 2.4G Keyboard Mouse
Bus 004 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

Baixar software necessário para utilização do gravador em :

<https://open-cells.com/d5138782a8739209ec5760865b1e53b0/uicc-v3.2.tgz>

Colocar na pasta de preferência e descompactar, da seguinte forma:

`tar -xvzf uicc-v3.2.tgz`

`cd uicc-v3.2`

`make`

```
root@incomm-ProLiant-ML110-Gen9:/home/ezequiel# cd Downloads/
root@incomm-ProLiant-ML110-Gen9:/home/ezequiel/Downloads# ls -la
.  ..  uicc-v3.2  uicc-v3.2.tgz
root@incomm-ProLiant-ML110-Gen9:/home/ezequiel/Downloads# tar -xvzf uicc-v3.2.tgz
uicc-v3.2/
uicc-v3.2/README.txt
uicc-v3.2/uicc.h
uicc-v3.2/milenage.h
uicc-v3.2/program_uicc.c
uicc-v3.2/aes.h
uicc-v3.2/program_uicc_pcsc
uicc-v3.2/Makefile
uicc-v3.2/program_uicc
root@incomm-ProLiant-ML110-Gen9:/home/ezequiel/Downloads# cd uicc-v3.2
root@incomm-ProLiant-ML110-Gen9:/home/ezequiel/Downloads/uicc-v3.2# make
make: 'program_uicc' está atualizado.
```


5. Configurando e executando

➤ Gravando os dados no simcard

Depois de inserir o simcard no gravador, basta executar inserindo dados:

```
./program_uicc --adm 0C012815 --imsi 0010100000000002 --key  
fec86ba6eb707ed08905757b1bb44b8f --opc BFD5771AAF4F6728E9BC6EF2C2533BDB
```

Obs.: os valores do imsi, key e opc podem ser diversos, por isso, não foi colocado nenhum valor. Contudo, o valor de adm costuma vir descrito e necessita ser alterado com risco de bloqueio do simcard em caso de erro sucessivos do valor. por isso, não foi colocado nenhum valor.

```
root@incomm-ProLiant-ML110-Gen9:/home/incomm/Downloads/uicc-v3.2# sudo ./program_uicc --adm 0C012816 --im  
si 0010100000000001 --key 82E9053A1882085FF2C020359938DAE9 --opc BFD5771AAF4F6728E9BC6EF2C2533BDB  
  
Existing values in USIM  
ICCID: 89860061100000000816  
WARNING: iccid luhn encoding of last digit not done  
USIM IMSI: 310789012345304  
USIM MSISDN: 00000816  
USIM Service Provider Name: open cells  
  
Setting new values  
  
Reading UICC values after uploading new values  
ICCID: 89860061100000000816  
WARNING: iccid luhn encoding of last digit not done  
USIM IMSI: 0010100000000001  
USIM MSISDN: 00000816  
USIM Service Provider Name: open cells  
root@incomm-ProLiant-ML110-Gen9:/home/incomm/Downloads/uicc-v3.2#
```

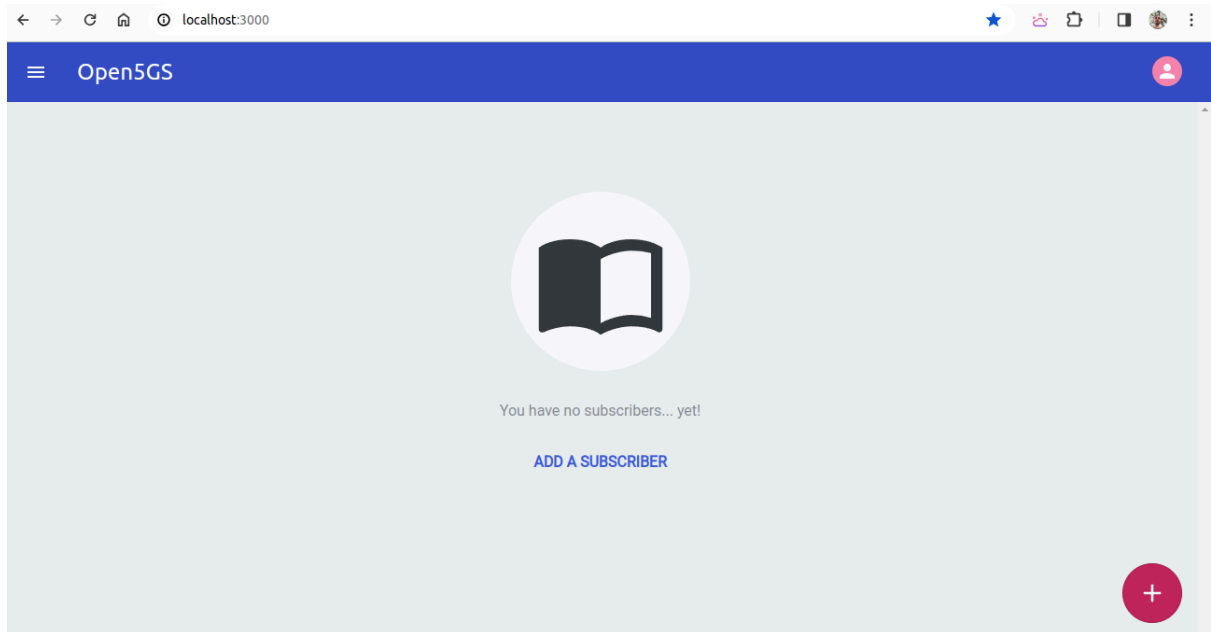
➤ Mudando os parâmetros do Open5GS

Conecte-se a <http://localhost:3000> e faça login com uma conta de administrador.

Username : admin
Password : 1423

Depois de logado:

- I. Vá para o menu do assinante;
- II. Clique no botão + para adicionar um novo assinante;
- III. Preencha o IMSI, (K, OPC, AMF) e APN do assinante e salve.

A screenshot of the 'Editar assinante' (Edit subscriber) form in the Open5GS web interface. The form is titled 'Configuração do assinante' (Subscriber configuration). It contains several input fields and dropdown menus. The 'IMSI*' field is filled with '001010000000001'. Below it is a blue button with a white plus sign. The 'Chave de Assinante (K)*' field is filled with '82E9053A1882085FF2C020359938DAE9'. The 'Campo de gerenciamento de autenticação (AMF)*' field is filled with '8000'. The 'Tipo USIM' dropdown menu is set to 'OPc'. The 'Chave do Operador (OPc/OP)*' field is filled with 'BFD5771AAF4F6728E9BC6EF2C2533BDB'. The 'Link descendente UE-AMBR*' field is filled with '1'. The 'Unidade' dropdown menu is set to 'Gbps'. The 'Ligação UE-AMBR*' field is filled with '1'. The 'Unidade' dropdown menu is set to 'Gbps'. At the bottom right of the form, there are two buttons: 'CANCELAR' and 'SALVAR'.

Modifique o `amf.yaml` e `upf.yaml` para definir o endereço IP S1AP, ID PLMN e TAC:

Padrão:

```
GNU nano 6.2                                anf.yaml
#   relative_capacity: 100
#
amf:
  sbt:
    - addr: 127.0.0.5
      port: 7777
  ngap:
    - addr: 127.0.0.5
  metrics:
    - addr: 127.0.0.5
      port: 9090
  guami:
    - plmn_id:
        mcc: 999
        mnc: 70
        amf_id:
            region: 2
            set: 1
  tai:
    - plmn_id:
        mcc: 999
        mnc: 70
        tac: 1
  plmn_support:
    - plmn_id:
        mcc: 999
        mnc: 70
        s_nssai:
            - sst: 1
  security:
    integrity_order : [ NIA2, NIA1, NIA0 ]
    ciphering_order : [ NEA0, NEA1, NEA2 ]

GNU nano 6.2                                upf.yaml
# <Metrics Server>
#
# o Metrics Server(http://<any address>;9090)
# upf:
#   metrics:
#     - addr: 0.0.0.0
#       port: 9090
#
upf:
  pfcp:
    - addr: 127.0.0.7
  gtpu:
    - addr: 127.0.0.7
  subnet:
    - addr: 10.45.0.1/16
    - addr: 2001:db8:cafe::1/48
  metrics:
    - addr: 127.0.0.7
      port: 9090

#
# <PFCP Client>>
#
# o PFCP Client(127.0.0.4;8805)
# smf:
#   pfcp:
#     addr: 127.0.0.4
#
smf:
#
# o Number of output streams per SCTP associations.
```

Modificado:

/etc/open5gs/nrf.yaml /etc/open5gs/nrf.yaml

```
GNU nano 6.2 upf.yaml *
# <Metrics Server>
#
# o Metrics Server(http://<any address>:9090)
# upf:
#   metrics:
#     - addr: 0.0.0.0
#       port: 9090
#
# upf:
#   pfcp:
#     - addr: 127.0.0.7
#   gtpu:
#     - addr: 127.0.1.100
#   subnet:
#     - addr: 10.45.0.1/16
#     - addr: 2001:db8:cafe::1/48
#   metrics:
#     - addr: 127.0.0.7
#       port: 9090
#
# <PFCP Client>
#
# o PFCP Client(127.0.0.4:8005)
# snf:
#   pfcp:
#     addr: 127.0.0.4
#
# snf:
#
# o Number of output streams per SCTP associations.
```

Obs.: foi utilizado o comando “nano” para fazer essa alteração e futuras, ainda assim, pode ser utilizado outros a depender da escolha de modificação, por exemplo, pode ser usado o vscode para não precisar ficar sempre no terminal do linux e poder voltar a versões anteriores do arquivo mais facilmente. Sugiro que use locate para encontrar o diretório adequado.

Após alterar os arquivos conf, reinicie os daemons Open5GS.

```
sudo systemctl restart open5gs-mmed
sudo systemctl restart open5gs-sgwud
```

Para fazer a ponte entre PGWU/UPF e WAN (Internet), você deve habilitar o encaminhamento de IP e adicionar uma regra NAT às suas tabelas de IP.

Habilitar encaminhamento IPv4/IPv6:

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo sysctl -w net.ipv6.conf.all.forwarding=1
```

Adicionar regra NAT:

```
sudo iptables -t nat -A POSTROUTING -s 10.45.0.0/16 ! -o ogstun -j MASQUERADE #
sudo ip6tables -t nat -A POSTROUTING -s 2001:db8:cafe::/48 ! -o ogstun -j
MASQUERADE
```

Configure o firewall corretamente:

```
sudo ufw status
```

Retornando “Status: active” use:

```
sudo ufw disable  
sudo ufw status
```

Retornando “Status: inactive” prossiga o tutorial..

Aviso: tu deverias considerar as configurações abaixo por motivos de segurança.

Certifique-se de que os pacotes na cadeia `INPUT` para a interface `ogstun` sejam aceitos:

```
sudo iptables -I INPUT -i ogstun -j ACCEPT
```

Impedir que os UEs se conectem ao host no qual o UPF está sendo executado:

```
sudo iptables -I INPUT -s 10.45.0.0/16 -j DROP  
sudo ip6tables -I INPUT -s 2001:db8:cafe::/48 -j DROP
```

Se a sua rede principal for executada em vários hosts, você provavelmente desejará bloquear UE originando tráfego de acesso a outras funções de rede. Substitua x.x.x.x/y pelo IP/sub-rede VNFs:

```
sudo iptables -I FORWARD -s 10.45.0.0/16 -d x.x.x.x/y -j DROP
```

➤ **Mudando os parâmetros do srsRAN_Project**

“Modifique o `amf.yaml` e `upf.yaml` para definir o endereço IP S1AP, ID PLMN e TAC:”

Padrão:

```
[OBJ]
```

Modificado:

```

GNU nano 6.2 gnb rf b200 tdd n78 20mhz.yml *
# This example configuration outlines how to configure the srsRAN Project gNB to create a single TDD cell
# transmitting in band 78, with 20 MHz bandwidth and 30 kHz sub-carrier-spacing. A USRP B200 is configured
# as the RF frontend using split 0. Note in this example an external clock source is not used, so the sync
# is not defined and the default is used.

amf:
  addr: 127.0.0.5 # The address or hostname of the AMF.
  bind_addr: 127.0.0.1 # A local IP that the gNB binds to for traffic from the AMF.

ru_sdr:
  device_driver: uhd # The RF driver name.
  device_args: type=b200,num_recv_frames=64,num_send_frames=64 # Optionally pass arguments to the selected RF driver.
  srates: 23.04 # RF sample rate might need to be adjusted according to selected bandwidth.
  otw_format: sc12
  tx_gain: 50 # Transmit gain of the RF might need to be adjusted to the given situation.
  rx_gain: 60 # Receive gain of the RF might need to be adjusted to the given situation.

cell_cfg:
  dl_arfcn: 632628 # ARFCN of the downlink carrier (center frequency).
  band: 78 # The NR band.
  channel_bandwidth_MHz: 20 # Bandwidth in MHz. Number of PRBs will be automatically derived.
  common_scs: 30 # Subcarrier spacing in kHz used for data.
  plmn: "00101" # PLMN broadcasted by the gNB.
  tac: 7 # Tracking area code (needs to match the core configuration).
  pci: 1 # Physical cell ID.

log:
  filename: /tmp/gnb.log # Path of the log file.
  all_level: warning # Logging level applied to all layers.

pcap:
  mac_enable: false # Set to true to enable MAC-layer PCAPs.
  mac_filename: /tmp/gnb_mac.pcap # Path where the MAC PCAP is stored.
  ngap_enable: false # Set to true to enable NGAP PCAPs.

```

Volte para o diretório de origem do srsRAN e modifique:

Obs.: utilizei o comando `locate` para encontrar o diretório adequado. Além disso, os dados padrões dos arquivos foram preservados em forma de comentário conforme demonstrado nas imagens anteriores, pois serão utilizados posteriormente.

➤ Mudando os parâmetros no celular:

Retire o SIM card da sua operadora e insira o de teste que está sendo utilizado para definir os parâmetros para que fique no slot 1 do celular e crie uma APN, use os dados encontrado em “http://localhost:3000” como referência para nome e protocolo.

6. Executando os binários e usando a rede configurada

Obs.: opcionalmente, tu podes instalar e abrir o wireshark para capturas de log.

```

sudo apt-get install wireshark
sudo wireshark

```

Selecion: `Loopbkacj:lo`

Nota 02: não é obrigatório executar cada um dos binários em terminais diferentes, porém é necessário usar pelo menos dois terminais distintos e suas janelas. Outrossim, cada um dos comandos abaixo necessita estar em um diretório específico. Portanto, use novamente o `locate` para encontrar o diretório correto.

```
./open5gs-nrfd  
./open5gs-scpd  
./open5gs-amfd  
./open5gs-smfd  
./open5gs-ausfd  
./open5gs-udmd  
./open5gs-udrdb  
./open5gs-pcfd  
./open5gs-nssfd  
./open5gs-bsfd  
./open5gs-upfd
```

```
gnb -c gnb_rf_b200_tdd_n78_20mhz.yml
```

Em caso de êxito, poderás usar a rede 5G criada:

